Gate 4 — Wiremock API Testing+ Selenide UI

- Participant: R.P.Cipiraj Date: ‹27.6.26 Branch: ‹cipiraj-week04-gate04>
- Assessment: Week 4 · Gate 4 —Wiremock API + Selenide UI Testing

1. Objective & user story

In this assessment,there are two jobs Firstly, that get the consumers request and expectation as a pact. Generate and Publish the pact to the Pact Broker.Then the Provider verifies the pact and provides the verification results.Based on the Verification results,deployment can be decided.

Secondly,we need to assert the visibility of the ‘availability’ badge in the cart page ,with no manual waits,no thread.sleep.

2. Environment & setup

Layer	Pinned choice
Language / build	Java 21 · Maven · Surefire 3.5.6
HTTP client	RestAssured 6.0.0
Test platform	IntelliJ,Github-Actions
Reporting	Screenshots as Evidence
OS / machine	Windows 11› · Java 21

Prerequisites & how to run

As we not using a Docker,need to create a account in pactflow.io and get the url and token:
PACT_BROKER_URL
PACT_BROKER_TOKEN
Selenide Depedency

To generate the pacts:

- mvn clean -Dtest=PosOmsConsumerPactTest test

3. Pact

3.1 Consumer - Pacts Generation

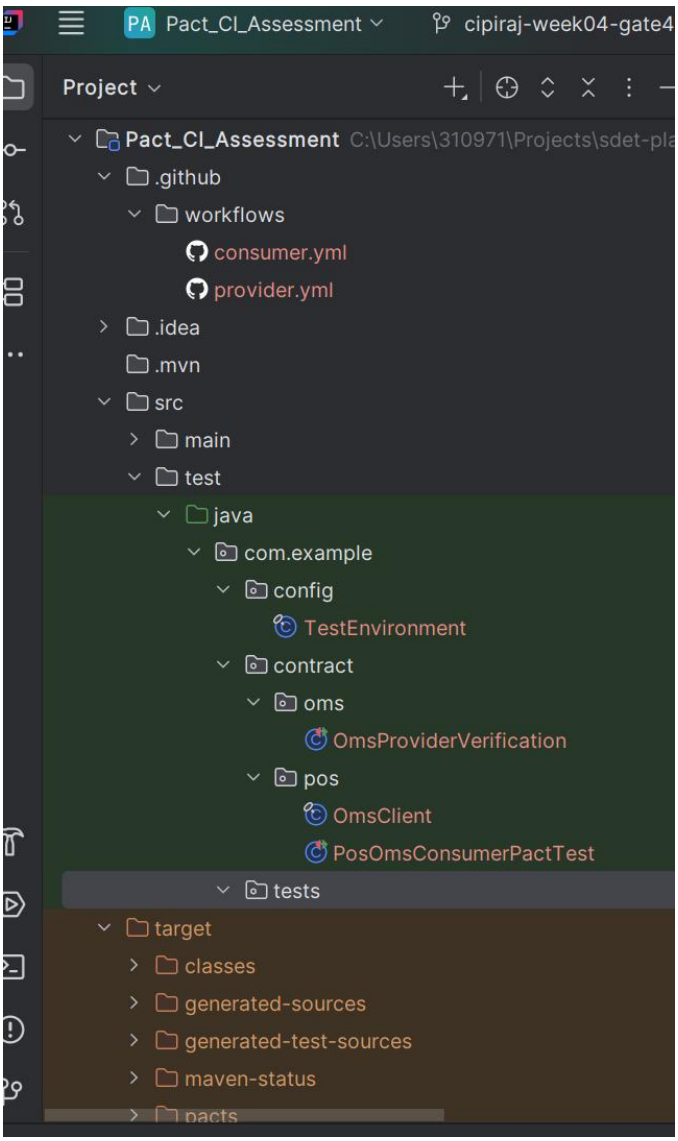
- The Generated Pacts are published to the pact broker
- In the assessment,using OmsClient pacts are getting generated and it is asserted

3.2 Provider- Verifies the pact

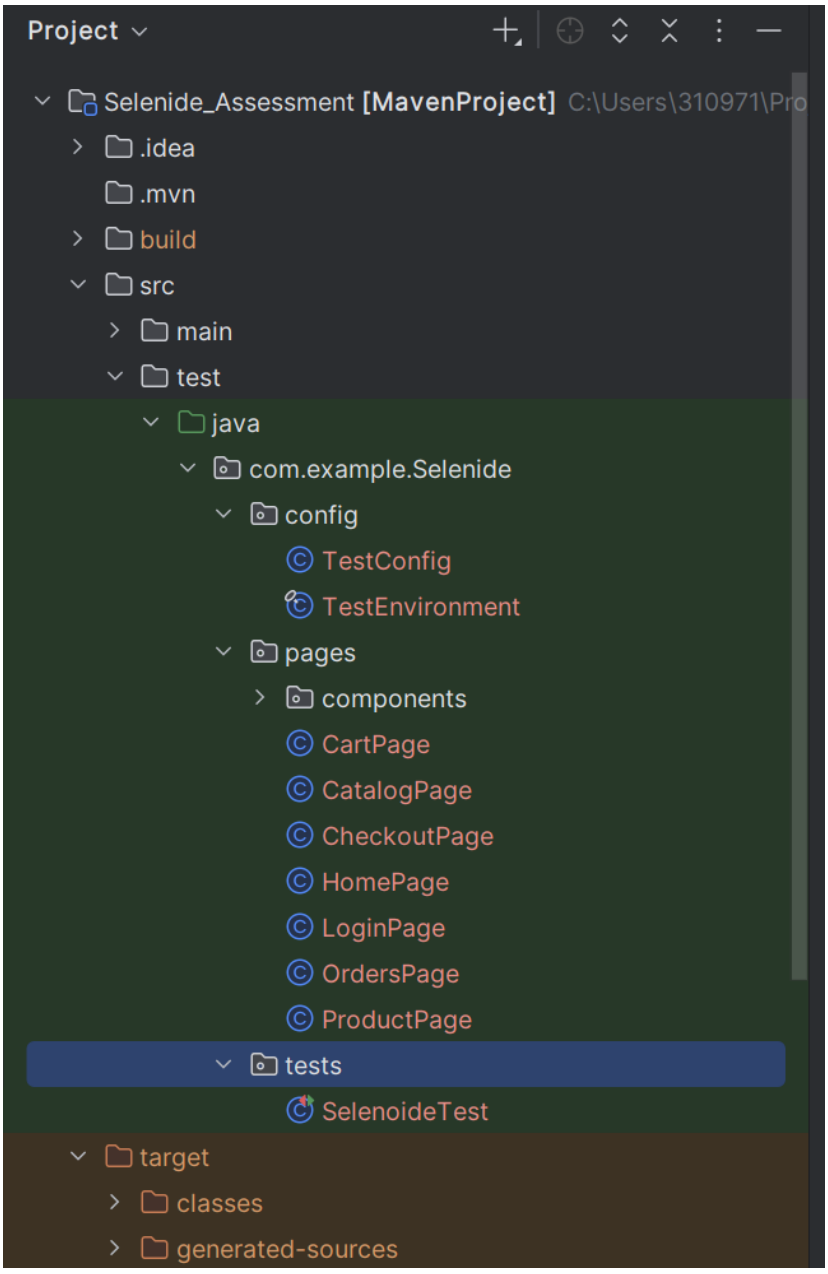
- After the pacts are getting generated,then the pact broker provides it to the provider for the verification

4. Project structure

Wiremock:



Selenide:



5. Test cases covered (3)

Wiremock:

1.

```
@Test
@PactTestFor(pactMethod = "getOrder")
void testGetOrder(MockServer mockServer) {

    OmsClient client =
            new OmsClient(mockServer.getUrl());

  OmsClient.Order order = client.getOrder( id: 123);

    assertEquals( expected: 200, order.statusCode());
    assertEquals( expected: 123, order.orderId());
    assertEquals( expected: "CONFIRMED", order.status());
    assertEquals( expected: 12.0, order.total());
}
```

2.

```
@Test
@PactTestFor(pactMethod = "getInvalidOrder")
void testInvalidGetOrder(MockServer mockServer) {

    OmsClient client =
            new OmsClient(mockServer.getUrl());

    OmsClient.InvalidOrder invalid =
            client.invalidgetOrder(9999);

    assertEquals( expected: 404, invalid.statusCode());
    assertEquals( expected: "Item Not Found", invalid.error());

}
}
```

3.In Selenide

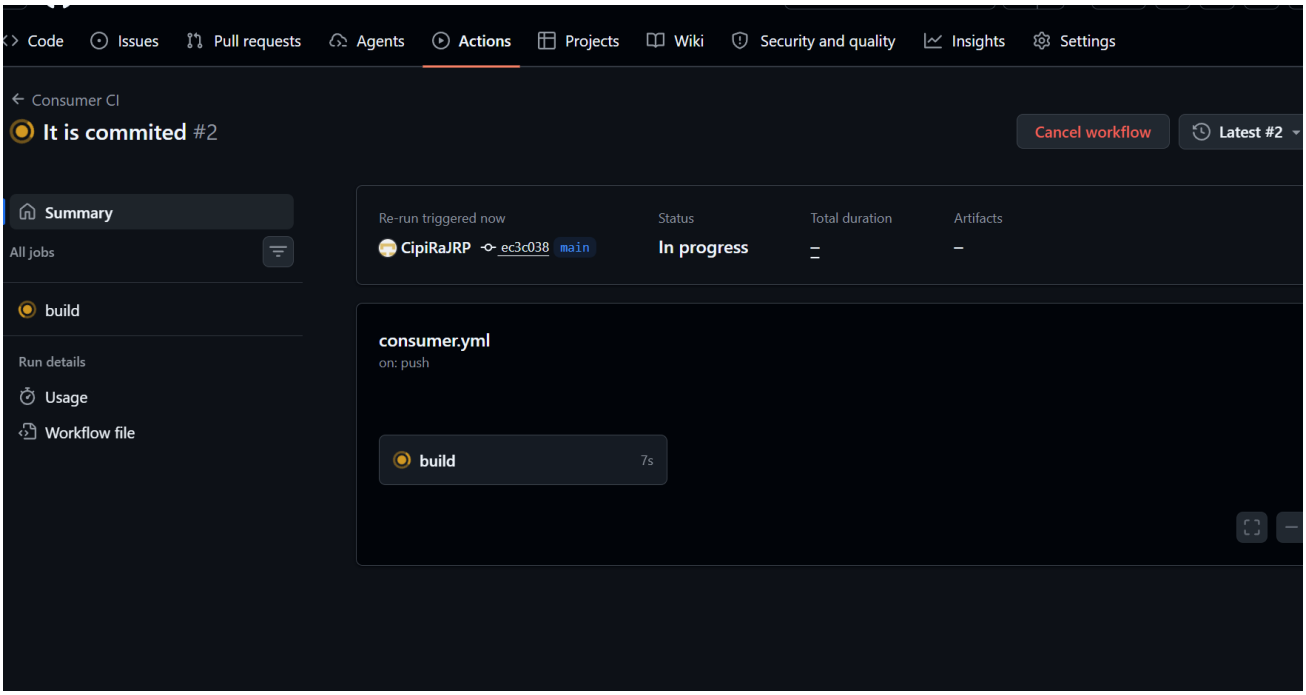
```
    static void setup() { TestConfig.apply(); }


    @Test
    @DisplayName("Module Test of Cart Verification")
    void checkAccessibility(){
        HomePage homePage = new HomePage()
                .gotohome();
        LoginPage loginPage = homePage.gotoLogin();
        homePage = loginPage.makeLogin(TestEnvironment.required( name: "EMAIL"),

        CatalogPage catalogPage = homePage.goToCatalogAsLoggedUser()
                .searchProduct(TestEnvironment.required( name: "PRODUCT_NAME"));
```

7. Results

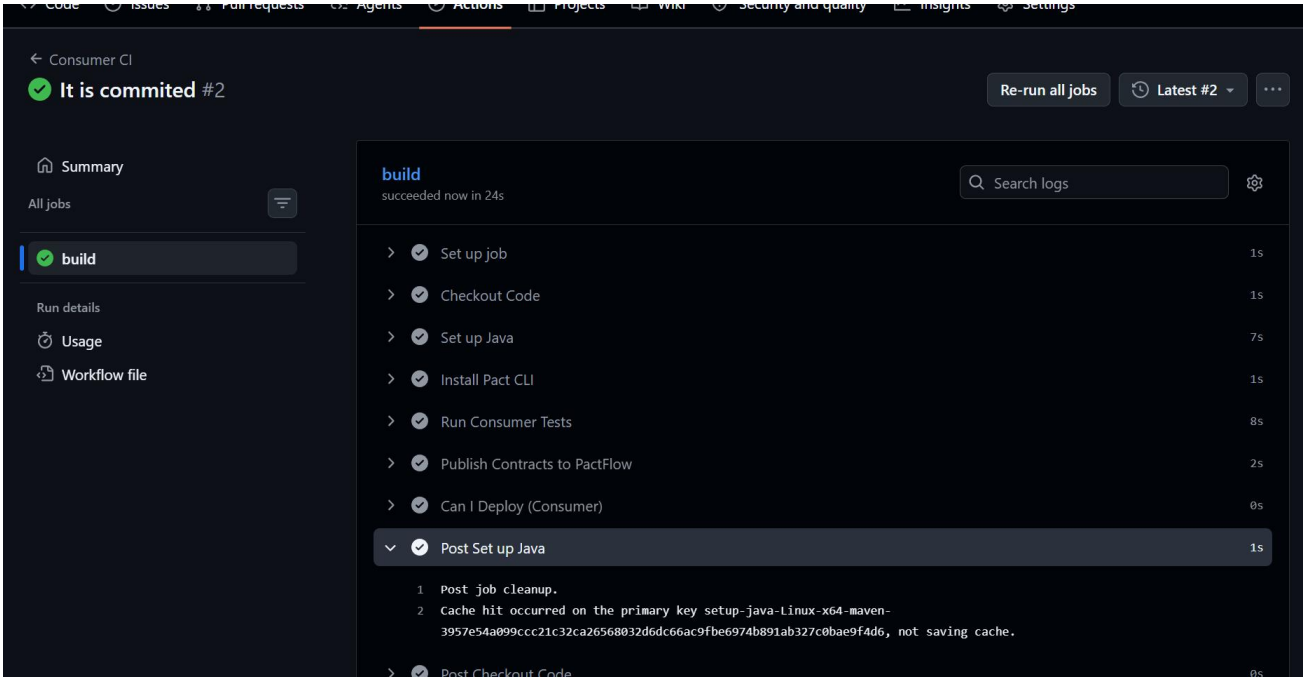
There are 3 Testcase:
In Wiremock asserting 200 and asserting 403
In Seledide asserting the visibility of an Element
All 3 Testcases passed

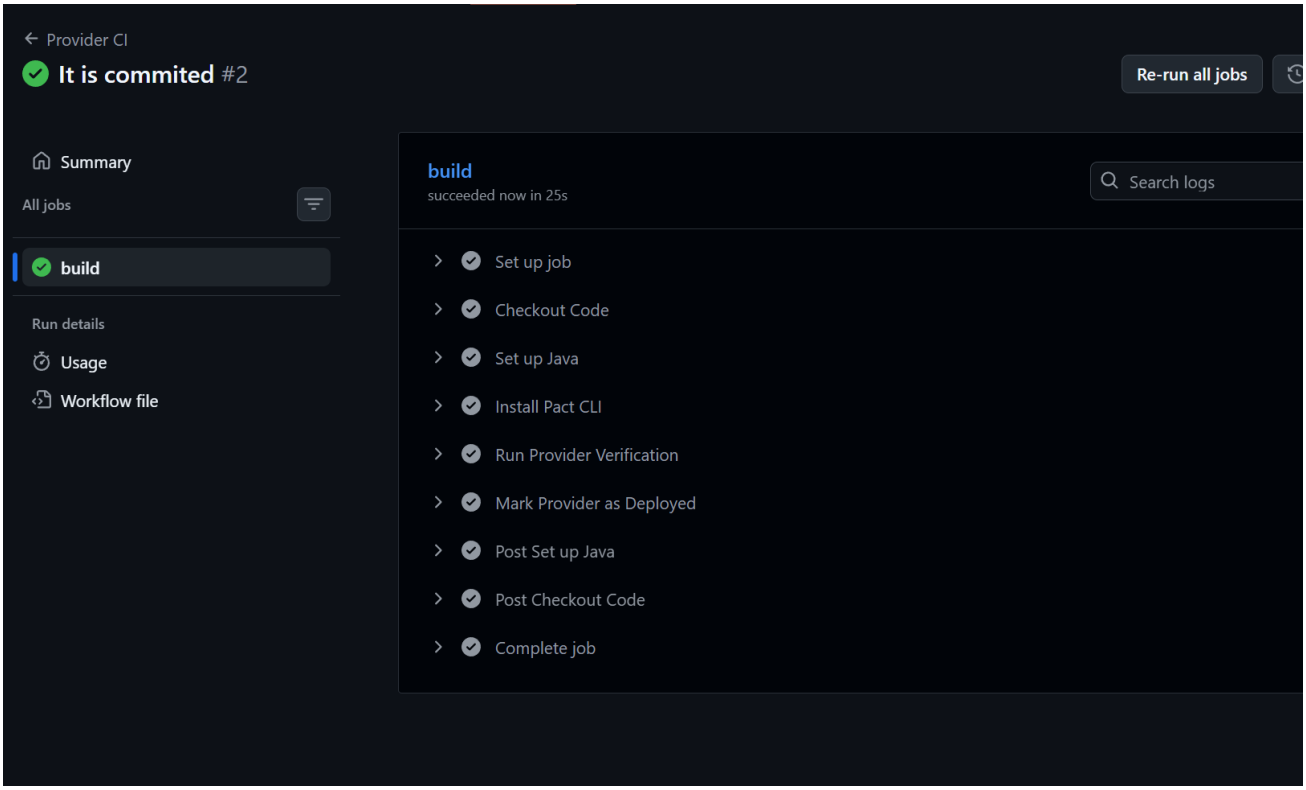
8. Evidence

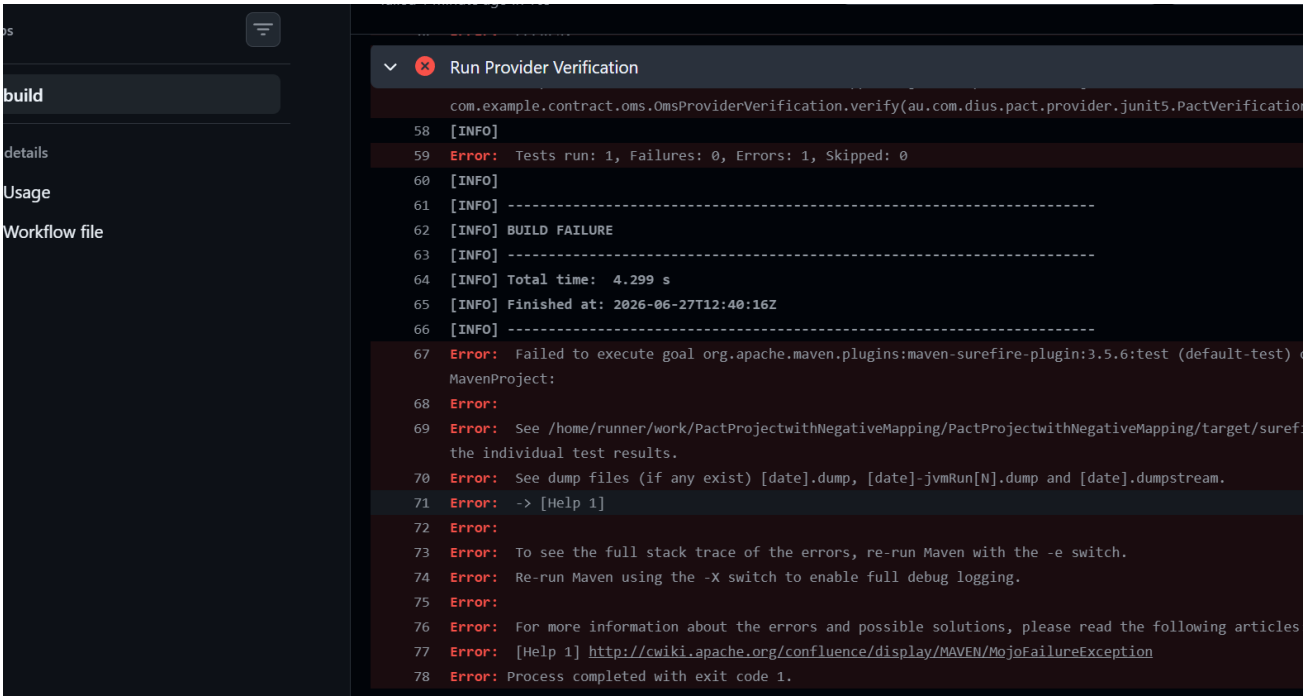
Wiremock:



Gate 4— Wiremock API and Selenide UI Testing



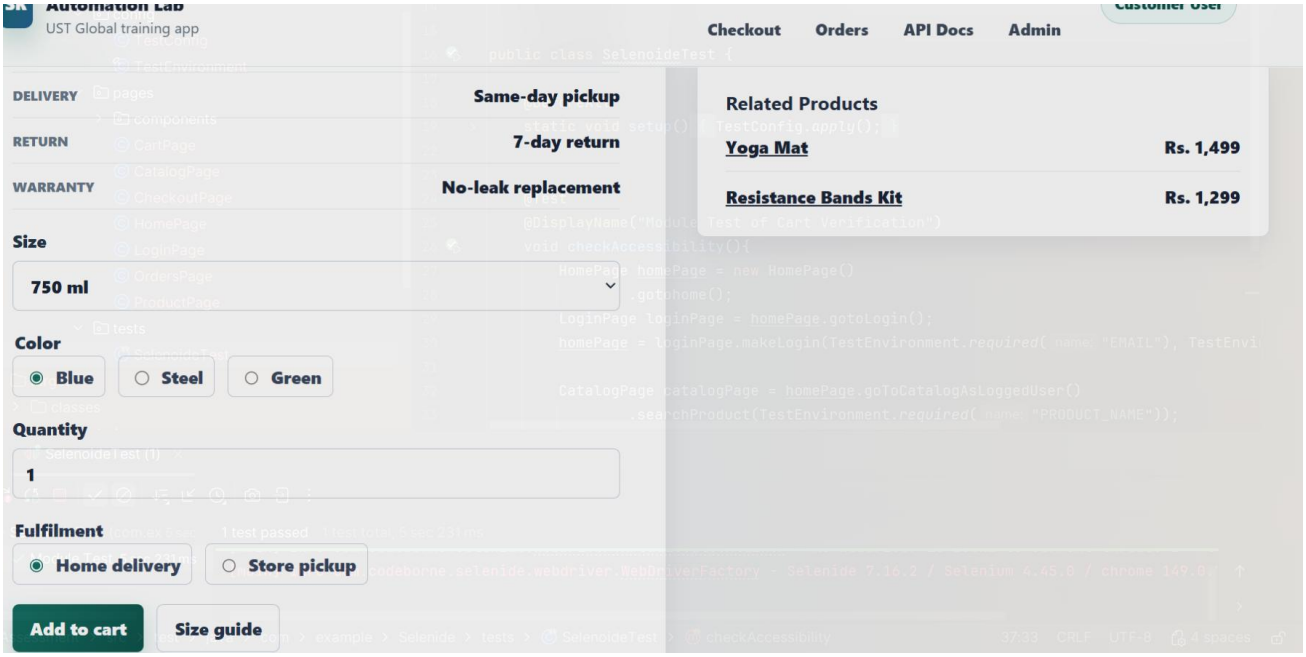




Broker API due to intentional failure done in Provider Side

```
                """)));
    }

    @State("Order 9999 does not exists")  no usages  Cipiraj Rangaraj Parimaladevi(UST
    void isOrderNotExists(){
        wireMock.stubFor(get(urlEqualTo( testUrl: "/orders/9999"))
                .willReturn(aResponse()
                        .withStatus(404)
                        .withHeader( key: "Content-Type", ...values: "application/json")
                        .withBody("""
                                {"status":"CONFIRMED"}
                                """)));
    }

}
```

Selenium



9. Github Repository

Github link: https://github.com/ust-sdet-training/SDET/tree/cipiraj-week04-gate4

10. Conclusion — what this proves

A Selenide Verification and Wiremock consumer and Provider verification is done by CI/CD